

## **EXPERIMENT NO: 3**

**ROLL NO:240701038**

**NAME:ANBU SELVAN V**

### **Survey Report on Emerging Trends in Low-Code Application Development**

#### **1. Aim:**

To study the emerging trends in **low-code application development**, understand how low-code platforms differ from traditional application development, analyze their benefits and challenges, and compare commonly used low-code platforms.

#### **2. Traditional Application Development vs Emerging Trends in Low-Code Application Development:**

##### **2.1 Traditional Application Development:-**

Traditional application development involves building applications mainly through manual coding. Developers write code for the interface, business logic, database, testing, and deployment. It provides high flexibility and customization but requires skilled programmers and generally takes more time and effort to develop and maintain

##### **2.2 Low-Code Application Development:-**

Low-code development uses **visual tools, drag-and-drop components, pre-built modules, and reusable elements** to create applications with minimal manual coding. It supports faster development and allows both professional developers and business users to participate in application development. Custom coding can still be used when advanced features are required.

## 2.3 Emerging Trends in Low-Code Application Development:-

The important emerging trends are:

- **AI-Assisted Development:** AI helps generate code, formulas, workflows, and application components.
- **Workflow Automation:** Business processes such as approvals, notifications, and record updates can be automated.
- **Citizen Development:** Non-professional users can build applications using visual development tools.
- **Cloud-Based Development:** Applications can be developed, deployed, and managed through cloud platforms.
- **API Integration:** Applications can easily connect with databases, APIs, and external services.
- **Reusable Components:** Templates and components can be reused to reduce development effort.
- **Rapid Prototyping:** Working prototypes can be created quickly and improved based on feedback.
- **Mobile Development:** Low-code platforms support responsive and mobile applications.
- **Data Analytics:** Reports, dashboards, and visualizations can be integrated into applications.

## 2.4 Summary

Traditional development depends mainly on manual coding, while low-code development emphasizes **visual development, automation, and reusable components**. With emerging technologies such as AI, cloud computing, API integration, and analytics, low-code platforms are making application development faster and more accessible.

### **3. Benefits and Challenges in Low-Code Development:**

#### **3.1 Benefits**

##### **1. Faster Development**

Drag-and-drop components and pre-built functionality reduce the amount of manual coding required.

##### **2. Reduced Development Effort**

Developers can use existing components, integrations, and workflows rather than implementing everything from scratch.

##### **3. Easy Prototyping**

A working prototype can be created quickly, making it easier to demonstrate and refine application requirements.

##### **4. Increased Productivity**

Developers can concentrate on complex business logic while the platform handles common application functions.

##### **5. Citizen Development**

Employees with limited programming knowledge can develop applications for simple business requirements.

##### **6. Workflow Automation**

Processes such as notifications, approvals, data validation, and record updates can be automated.

##### **7. Easier Maintenance**

Visual components and centralized platform features can simplify application maintenance.

##### **8. Integration Support**

Many low-code platforms provide ready-made connectors for databases, APIs, cloud services, and business applications.

## **3.2 Challenges**

### **1. Limited Customization**

Highly specialized requirements may be difficult to implement if they are not supported by the platform.

### **2. Vendor Lock-in**

Applications can become dependent on a particular platform's features, pricing, and infrastructure.

### **3. Security Concerns**

Applications must be configured carefully to protect sensitive information and control user access.

### **4. Performance Limitations**

Very complex or high-scale applications may require more control than a low-code platform provides.

### **5. Learning the Platform**

Although coding requirements are reduced, users still need to understand the platform's components, workflows, and rules.

### **6. Cost**

Advanced features, integrations, storage, and larger numbers of users may require paid plans.

### **7. Dependency on Platform Updates**

Changes made by the platform provider can affect application functionality.

### **8. Complex Applications May Still Need Coding**

Low-code does not completely eliminate programming. Custom functions or advanced requirements may still require conventional code.

#### 4. Comparison of Existing Low-Code Platforms:

| Feature              | Zoho Creator                                | Microsoft Power Apps             | OutSystems                           | Mendix                               | Salesforce Platform               |
|----------------------|---------------------------------------------|----------------------------------|--------------------------------------|--------------------------------------|-----------------------------------|
| Developer            | Zoho                                        | Microsoft                        | OutSystems                           | Mendix                               | Salesforce                        |
| Development Approach | Visual + low-code                           | Low-code + visual                | Low-code                             | Low-code                             | Low-code                          |
| Form Building        | Drag-and-drop                               | Visual controls                  | Visual UI development                | Visual development                   | Visual development                |
| Workflow Automation  | Supported                                   | Supported through Power Automate | Supported                            | Supported                            | Supported                         |
| Database             | Built-in application database               | Dataverse and connectors         | Built-in/data integration options    | Supported                            | Salesforce data platform          |
| API Integration      | Supported                                   | Extensive connectors             | Supported                            | Supported                            | Supported                         |
| Mobile Development   | Supported                                   | Supported                        | Supported                            | Supported                            | Supported                         |
| AI Features          | AI-assisted capabilities                    | AI/Copilot capabilities          | AI-assisted development capabilities | AI-assisted development capabilities | AI-assisted capabilities          |
| Custom Coding        | Deluge and other supported options          | Power Fx and extensions          | Supported                            | Supported                            | Apex and other development tools  |
| Main Strength        | Business applications and rapid development | Microsoft ecosystem integration  | Enterprise application development   | Enterprise low-code development      | CRM and business applications     |
| Typical Users        | Businesses, developers, citizen developers  | Businesses and developers        | Enterprise teams                     | Enterprise teams                     | CRM/business users and developers |

## 5.Application used in the Experiment :

**Platform:** Zoho Creator

**Application:** Patient Appointment & Scheduling Management

**Form:** Appointment\_Booking\_Form

**Report:** Appointment\_Booking\_Report

The application uses the **13 Basic Fields** in Zoho Creator to collect patient personal details, contact information, medical details, preferred appointment schedule, and treatment consent. The submitted information is stored in the appointment report, allowing the front desk to review and manage appointment requests.

### FORM:

The screenshot displays the 'Appointment booking form' within the Zoho Creator application. The interface includes a dark sidebar on the left with navigation options: 'Patient appointment booking', 'Appointment booking form', 'Appointment booking form Report', and 'Appointment booking form'. The main form area contains the following fields:

- Patient\_Name:** Split into 'First Name' and 'Last Name' text boxes.
- Age:** A text box containing '#####'.
- Gender:** Radio button options for 'Male', 'Female', and 'Others'.
- Email Address:** A text box.
- Contact Number:** A text box with a country code dropdown set to '+91' and the number '81234 56789'.
- Residential Address:** A text box labeled 'Address Line 1'.
- Referred By:** A text box.
- Department:** A dropdown menu with '-Select-'.
- Existing Conditions:** A dropdown menu with '-Select-'.
- Symptoms Description:** A large text area.
- Preferred Appointment Date:** A text box with the format 'dd-MMM-yyyy'.
- Preferred Appointment Time:** A text box with the format 'HH:mm:ss'.
- consent for treatment:** A checkbox labeled 'I consent to examination and treatment'.

A 'Submit' button is located at the bottom of the form. A notification banner at the top right of the form area states: 'To exit full screen, press and hold Esc'. The top of the application window shows the title 'Patient appointment booking' and status links: 'Trial has expired', 'Upgrade', 'Edit this application', and 'Help'.

Patient appointment booking

Appointment booking form

Appointment booking form Report

Appointment booking form

To exit full screen, press and hold Esc

Patient Name

DHONI

MS

First Name

Last Name

Age

42

Gender

Male

Female

Others

Email Address

msdhoi07@gmail.com

Contact Number

+91

1234567890

Residential Address

07,ranchi

Address Line 1

Referred By

self

Department

General medicine

Existing Conditions

headache

Symptoms Description

Mild

Preferred Appointment Date

17-Sep-2026

Preferred Appointment Time

17:52:51

consent for treatment

☒ I consent to examination and treatment

Submit

REPORT:

Patient appointment booking

Appointment booking form

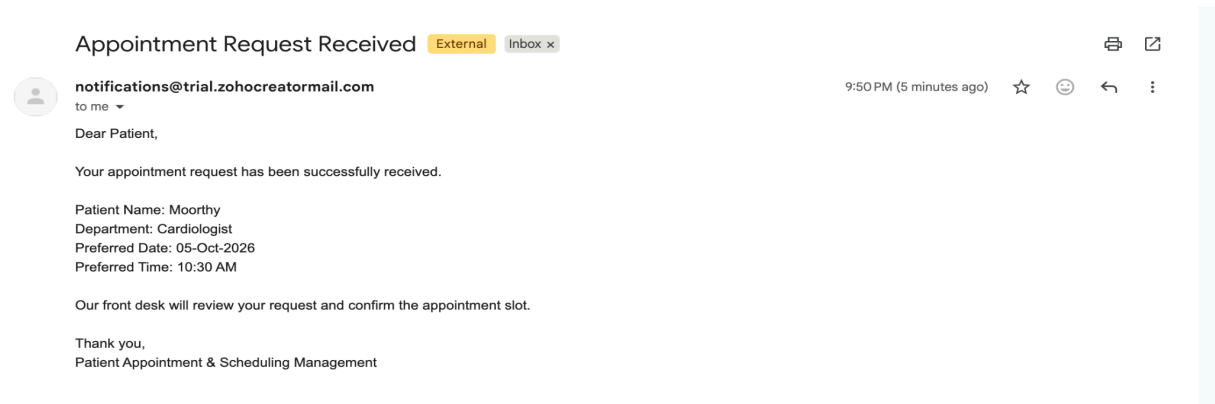
Appointment booking form Report

Appointment booking form Report

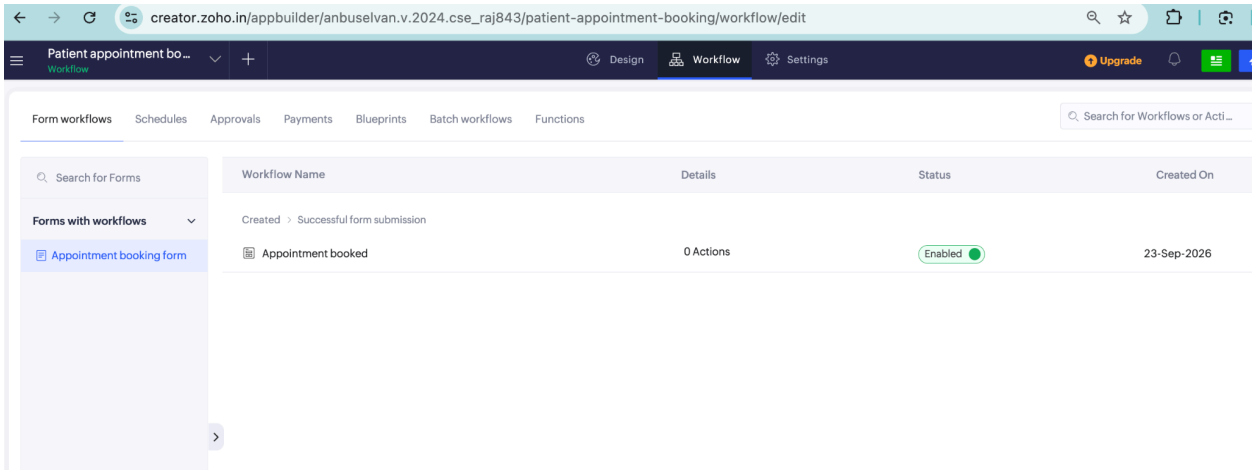
Showing 3 of 3

| Patient Name | Age | Gender | Email Address           | Contact Number | Residential Address | Referred By |
|--------------|-----|--------|-------------------------|----------------|---------------------|-------------|
| Rohit Sharma | 45  | Male   | rohitsharma45@gmail.com | +9178894561232 | 45,mumbai           | ritika      |
| Virat kohli  | 18  | Male   | viratkohli18@gmail.com  | +917894561230  | 18,bengaluru        | Anushka     |
| DHONI MS     | 42  | Male   | msdhoi07@gmail.com      | +911234567890  | 07,ranchi           | self        |

EMAIL:



WORKFLOW:



5. Result

The survey on emerging trends in low-code application development was **completed successfully**. The low-code development identified major trends such as AI-assisted development, workflow automation, cloud-based development, API integration, rapid prototyping, mobile development. A comparison of existing low-code platforms such as **Zoho Creator**, Microsoft Power Apps, OutSystems, Mendix, and Salesforce Platform was also studied based on their development approach, workflow automation, database support, API integration, AI features, custom coding, and major strengths.